

UXL SKILLS EVALUATOR

oneTBB Skill

Parallel patterns, flow graphs, arenas, and evidence-backed tuning

September 2026 • 7 of 7 evaluation tasks implemented

Why this is useful to oneTBB at first glance

- **Match problems to composable patterns** — Agents choose loops, reductions, scans, pipelines, flow graph, task groups, or arenas by workload shape.
- **Expose concurrency contracts** — Mutable state, ordering, cancellation, exceptions, blocking calls, and external pools are named before parallelizing.
- **Bound resource ownership** — Flow-graph admission, stage concurrency, and capacity return become correctness requirements.
- **Tune only from evidence** — Grainsize, affinity, arenas, and concurrency are measured against a serial reference and real CPU limits.

What the skill tells an agent today

- **Select the smallest correct oneTBB pattern** — Use the semantic shape of the work instead of writing explicit threads.
- **Make state and ordering explicit** — Partition-local temporaries beat shared mutation; join-node semantics must match correlation needs.
- **Control nested runtimes** — task_arena/global_control integrate oneTBB with thread pools, OpenMP, and other runtimes.
- **Treat failures as first-class** — Tests cover races, long-run resource bounds, cancellation, exceptions, duplicates, gaps, and ordering.
- **Keep GPU work elsewhere** — Device offload routes to SYCL or oneDPL guidance.

Evaluation inventory: all 7 tasks implemented

- **Patterns** — Histogram local aggregation; stable compaction scan; bounded image flow graph.
- **Runtime composition** — Nested thread-pool arena; cancellation and exception propagation.
- **Tuning** — Grainsize and affinity regression with measurable controls.
- **Incident evidence** — Join-node ordering is sourced from a maintainer incident.
- **What remains** — Human ownership and matched promotion trials — not more task-count filling.

How we chose the content

- **Official project guidance** — API docs, examples, tests, migration notes, scheduler behavior, and ownership files.
- **Failure-shaped coverage** — Shared state, unbounded flow graphs, oversubscription, affinity folklore, cancellation, and message correlation.
- **Executable correctness** — Each task has deterministic success criteria; two tasks retain measured quality headroom.
- **Clear boundaries** — The skill does not replace the API reference or claim GPU-offload coverage.

What project ownership would look like

- **Review the pattern-selection and runtime guidance** — Especially flow-graph backpressure, nested parallelism, join semantics, affinity, and migration.
- **Keep examples grounded** — Replace generic advice with project-supported idioms and tests where possible.
- **Calibrate, do not inflate** — The suite is already full; focus next on maintainer approval and whether the tasks discriminate.
- **UXL keeps the machinery** — Harbor, dashboards, matched comparisons, hosted CI, and shared validation remain centralized.

A focused 30-minute maintainer review

- **1. Confirm scope** — Are pattern selection, flow graph, arenas, cancellation, grainsize, affinity, and oversubscription the right first slice?
- **2. Correct one risky statement** — Especially legacy migration, scheduler semantics, or affinity/NUMA implications.
- **3. Judge the seven tasks** — Identify which are smoke coverage and which should measure real skill lift.
- **4. Name an owner** — A small reviewer group is enough; the infrastructure burden remains with UXL.
- **Outcome** — The strongest current portfolio becomes project-backed and ready for promotion calibration.